CSC 3320: System-Level Programming

Georgia State University

Chapter 17: Advanced Uses of Pointers (contd.)

Instructor: Dr. Md Mahfuzur Rahman
Department of Computer Science, GSU

Note: These slides are updated based on original notes from Raj Sunderraman, Michael Weeks, and Yuan Long

calloc() Example

Georgia State University

```c
int main() {
  int n = 5;
  int *ptr;

  // use calloc function to allocate the memory
  ptr = (int*)calloc(n, sizeof(int));

  if (ptr == NULL) {
      fprintf(stderr, "Memory allocation failed!\n");
      return 1;
  }

  //Display the array value
  printf("Array elements after calloc: ");
  for (int i = 0; i < n; i++) {
      printf("%d ", ptr[i]);
  }
  printf("\n");

  //free the allocated memory
  free(ptr);
  return 0;
}
```

Output: Array elements after calloc: 0 0 0 0 0

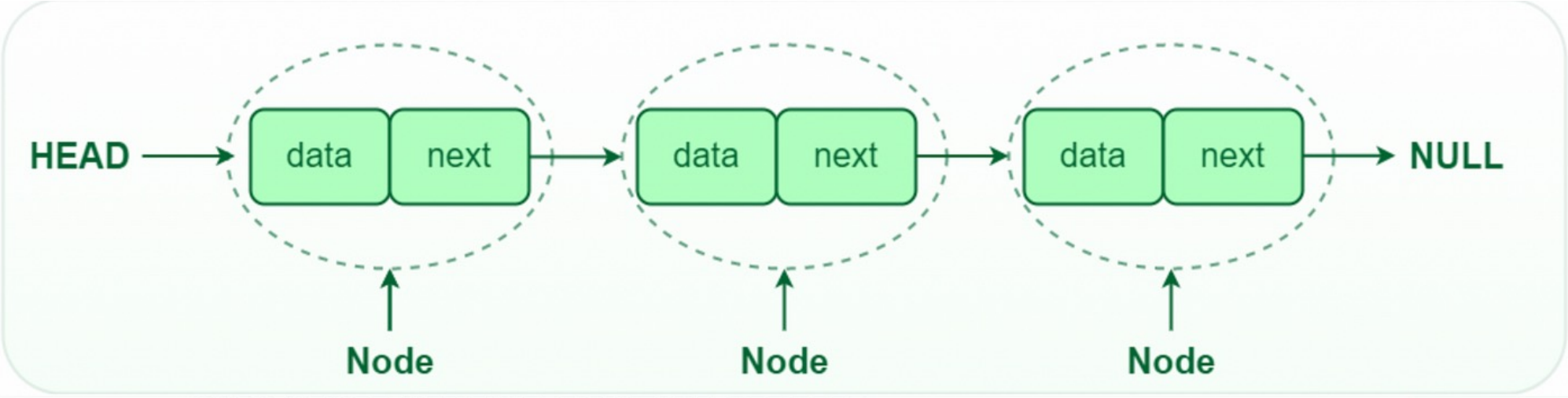
realloc() Example

Georgia State University

```c
int main() {
  // Initially allocate memory for 2 integers
  int *ptr = malloc(2 * sizeof(int));
  if (ptr == NULL) {
      fprintf(stderr, "Memory allocation failed\n");
      return 1;
  }

  ptr[0] = 5; ptr[1] = 10;

  // increase the size of the integer
  int *ptr_new = realloc(ptr, 3 * sizeof(int));
  if (ptr_new == NULL) {
      fprintf(stderr, "Memory reallocation failed\n");
      free(ptr);
      return 1;
  }
  ptr_new[2] = 15;

  int i;
  // Display the array
  for (i = 0; i < 3; i++) {
      printf("%d ", ptr_new[i]);
  }

  free(ptr_new);
  return 0;
}
```

Linked List Example

```c
#include <stdio.h>
#include <stdlib.h>


// Define the structure for a node in the linked list
struct Node {
    int data;
    struct Node *next;
};
```



Linked List Example

```
// Create a node, maintain a head
struct Node *head = NULL; // head points to the start node
struct Node *current = NULL; //current points to the latest created node
struct Node *new = NULL; // new points to the newly created node

printf("Add a record:\");
new = (struct Node *)malloc(sizeof(struct Node));
printf("\nEnter your data:");
scanf("%d", &new->data);
new->next = NULL;
head = current = new;

//whenever we want to create a new node, we can use the following:

printf("Add another record:\");
new = (struct Node *)malloc(sizeof(struct Node));
printf("\nEnter your data:");
scanf("%d", &new->data);
new->next = NULL;
current->next = new;
current = current->next;
```

Traversing a linked list

```
// Traverse the linked list and print the data
    current = head;
    while (current != NULL) {
        printf("%d ", current->data);
        current = current->next;
    }
```

Deallocating Storage

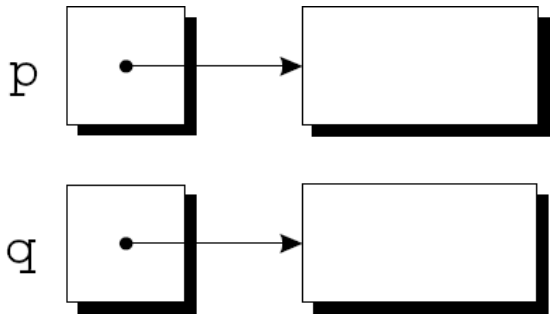
- `malloc` and the other memory allocation functions obtain memory blocks from a storage pool known as the ***heap.***
- Calling these functions too often—or asking them for large blocks of memory—can exhaust the heap, causing the functions to return a null pointer.
- To make matters worse, a program may allocate blocks of memory and then lose track of them, thereby wasting space.

Deallocating Storage

- Example:

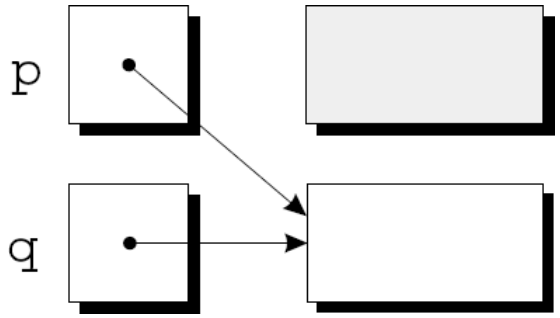
```
p = malloc(…);
q = malloc(…);
p = q;
```

- A snapshot after the first two statements have been executed:



Deallocating Storage

- After `q` is assigned to `p`, both variables now point to the second memory block:



- There are no pointers to the first block, so we'll never be able to use it again.

Deallocating Storage

Georgia State University

- A block of memory that's no longer accessible to a program is said to be ***garbage.***
- A program that leaves garbage behind has a ***memory leak.***
- Some languages provide a ***garbage collector*** that automatically locates and recycles garbage, but C doesn't.
- Instead, each C program is responsible for recycling its own garbage by calling the `free` function to release unneeded memory.

Deallocating Storage

- Memory allocated with malloc must be released after you are done with them.
- this memory is released by calling the function **free().**
- ***free*** expects as an argument the pointer returned by *malloc*.

- **free(ptr);**

Example

```
char *p = malloc(4);
…
free(p);
…
strcpy(p, “abc”); // WRONG: p is a dangling pointer
```

free(p);
p = NULL; // Safe practice is to set p to NULL after free()

Dangling Pointer Example


```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p = malloc(sizeof(int));
    *p = 42;

    printf("%d\n", *p);
    free(p);   // memory is deallocated
    *p = 99;    // Undefined behavior

    printf("%d\n", *p);

    /* Try allocating some memory. It may take up the freed space */
    int *q = malloc(sizeof(int));
    *q = 22;

    printf("%d\n", *p); // Garbage or irrelevant data

    return 0;
}
```

Freeing a linked list


```
void freeList(struct Node* head) {
    struct Node* current = head;
    struct Node* nextNode;

    while (current != NULL) {
        nextNode = current->next; // Save the next node
        free(current);            // Free the current node
        current = nextNode;       // Move to the next node
    }
}
```